

# Android Application Architecture (part.01 – Data Layer)

↑ 우리가 room을 사용해서  
만든 부분



# 목차

---

 Android App. Architecture 개요

 구현 준비

 Data Layer 구성

 전체 Architecture

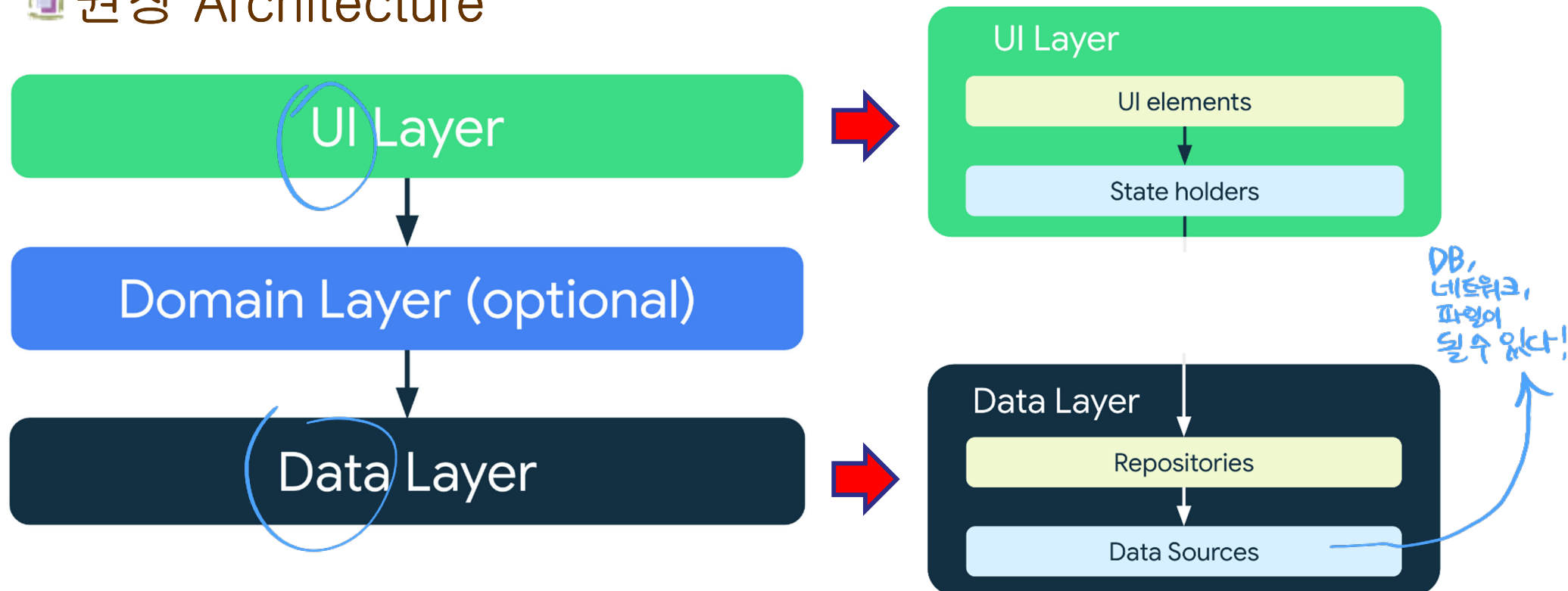
 참고사항

# Android App. Architecture 개요

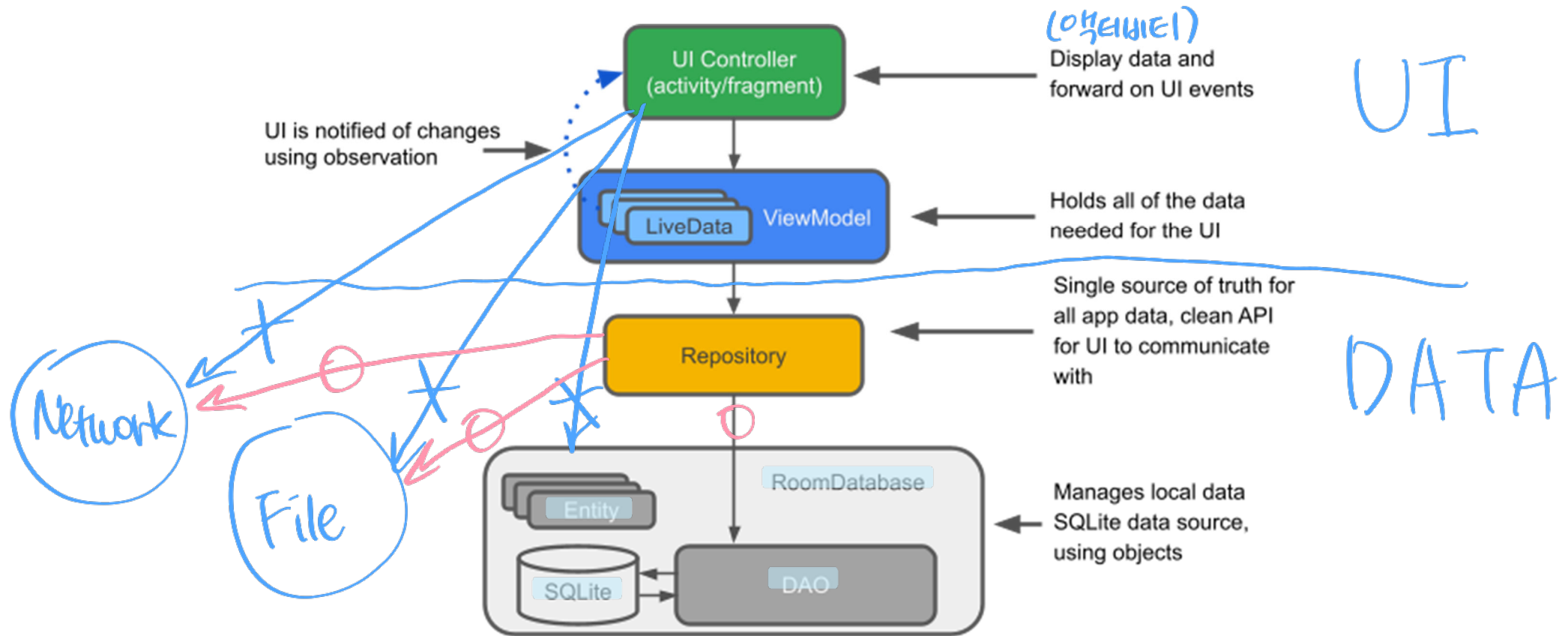
## 아키텍처 구성의 원칙

- ◆ 관심사의 분리 (Separation of Concerns) → *자신의 기능만!*
- ◆ 데이터 모델과 UI의 분리
- ◆ Data의 단일 소스 저장소 (SSOT: Single Source of Truth) 사용
- ◆ 단방향 데이터 흐름 (UDF: Unidirectional Data Flow)

## 권장 Architecture



# Architecture 주요 구성요소

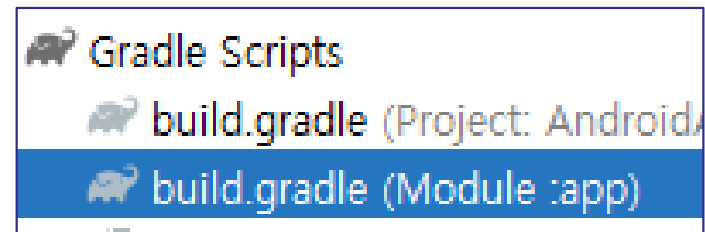


- LiveData : 데이터의 변경을 관찰할 수 있는 데이터 보관 클래스
- ViewModel : Data 와 UI 사이의 통신을 담당 및 데이터 지속 보유
- Repository: 데이터 소스를 관리하는 개발자 구현 클래스
- Entity : ROOM 사용 시 DB 테이블 표현 및 데이터 보관
- RoomDatabase : 데이터베이스 관리, DAO 생성
- DAO : 데이터 접근 객체
- SQLite DB : 데이터 보관

출처: <https://developer.android.com/topic/architecture?hl=ko>

## build.gradle (Module: ...)

```
plugins {  
    ...  
    id("com.google.devtools.ksp") version "1.9.0-1.0.13"  
}  
  
...  
  
dependencies {  
    ...  
  
    // ROOM  
    val room_version = "2.6.1"  
    implementation("androidx.room:room-runtime:$room_version")  
    annotationProcessor("androidx.room:room-compiler:$room_version")  
  
    // To use Kotlin Symbol Processing (KSP)  
    ksp("androidx.room:room-compiler:$room_version")  
  
    // optional - Kotlin Extensions and Coroutines support for Room  
    implementation("androidx.room:room-ktx:$room_version")  
  
    // Lifecycle components  
    val lifecycle_version = "2.8.5"  
    implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:$lifecycle_version")  
    implementation("androidx.lifecycle:lifecycle-livedata-ktx:$lifecycle_version")  
    implementation("androidx.lifecycle:lifecycle-common-java8:$lifecycle_version")  
}
```

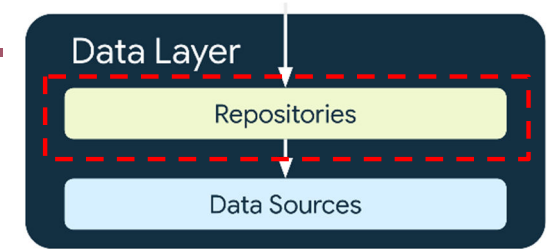


UI

# Data Layer 구성

☞ 데이터 소스를 포함하는 저장소로 구성

- ◆ 앱의 데이터를 한 곳에 모아 관리



## Repository

- ◆ 데이터 소스를 관리하는 클래스
- ◆ 예: FoodDao 를 관리하는 FoodRepository

```
class FoodRepository(private val foodDao: FoodDao) {
    val allFoods : Flow<List<Food>> = foodDao.getAllFoods()

    suspend fun addFood(food: Food) {
        foodDao.insertFood(food)
    }

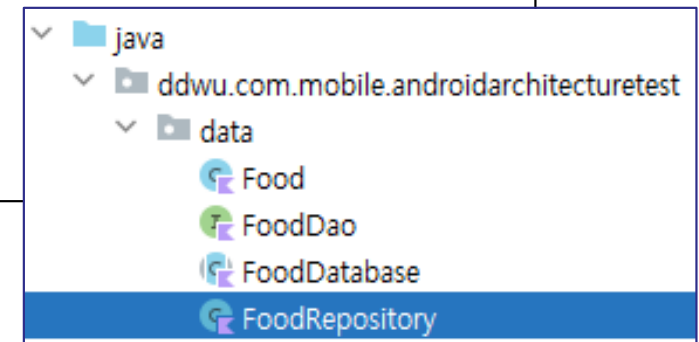
    ...
}
```

코루틴 적용을  
위해 suspend  
적용

FoodDao 를 private  
멤버로 선언

원샷 또는 관찰가능한 쿼리  
관련 멤버함수들 추가

필요한 DAO 함수 추가



# Repository 객체 사용 1

## Application 클래스 구현

### ◆ 앱을 표현하는 클래스

- 앱이 실행되는 동안 모든 구성요소에서 참조할 변수를 소유하도록 구현할 수 있음

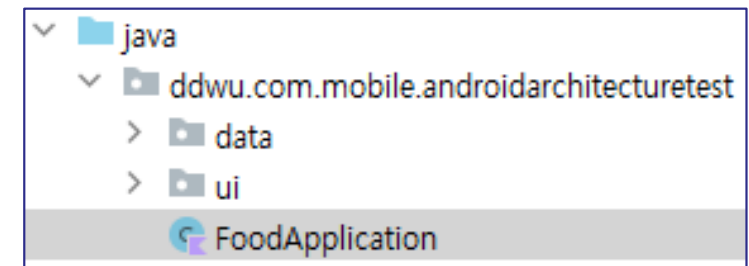
### ◆ Application 상속 클래스 구현 후 AndroidManifest에 등록

### ◆ 예: 앱 실행 중 계속 사용 가능한 FoodRepository

```
class FoodApplication: Application() {
    val foodDatabase by lazy {
        FoodDatabase.getDatabase(this)
    }

    val foodRepo by lazy {
        FoodRepository(database.foodDao())
    }
}
```

Application 상속



### ◆ FoodRepository 는 DB가 필요한 여러 구성요소(액티비티 등)에서 사용할 수 있으므로 Application의 멤버변수로 선언

# Repository 객체 사용 2

## 구현 Application 으로 기본 Application 대체

- ◆ AndroidManifest 에 추가한 Application 지정

```
<application
    android:name=".FoodApplication"
    android:allowBackup="true"
```

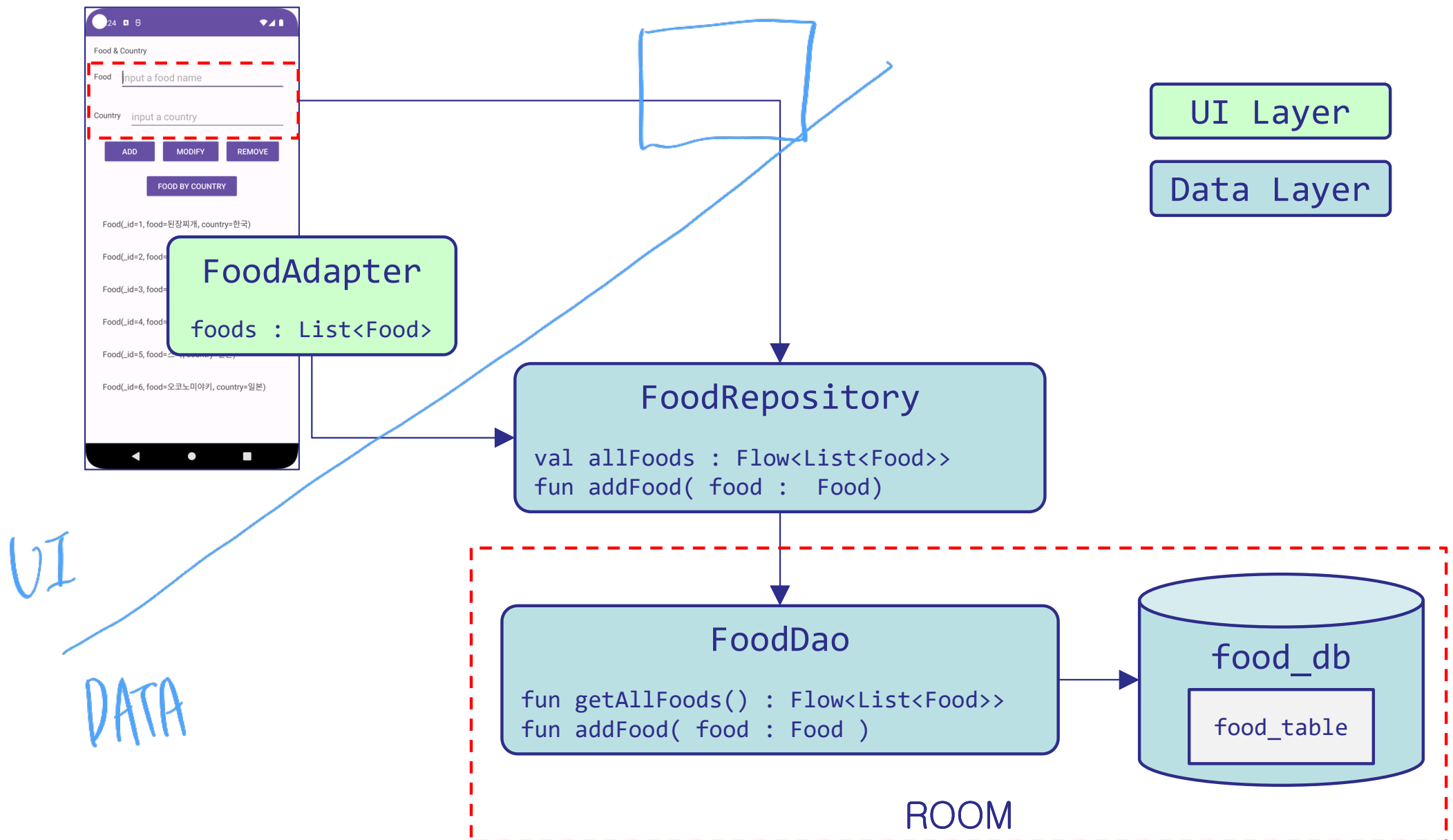
기본 Application에서 구현한 FoodApplication으로 대체

## Repository의 사용

- ◆ application 객체(시스템이 생성)의 멤버변수에 접근
- ◆ Application 객체를 구현 Application 객체로 형변환 필요
- ◆ 예: FoodRepository 의 allFoods 사용

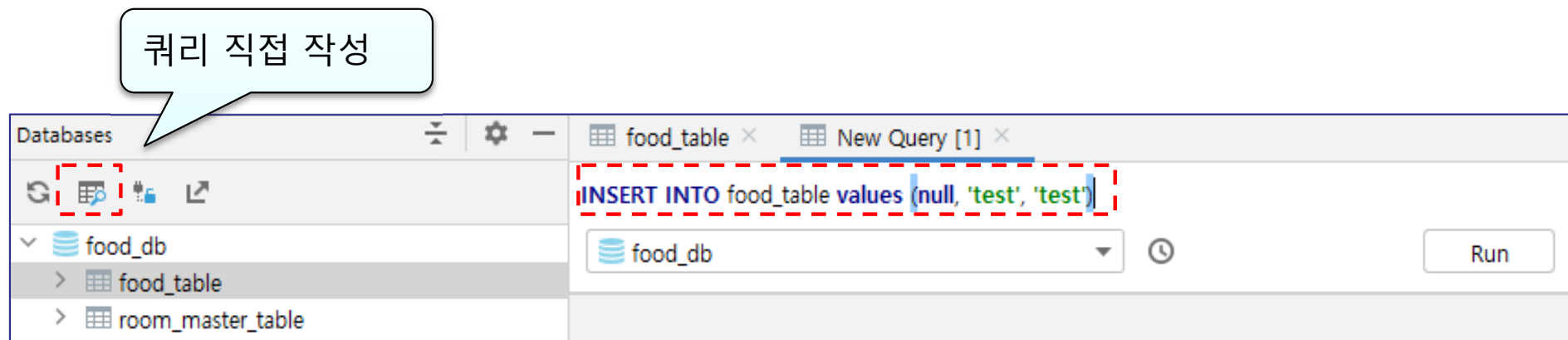
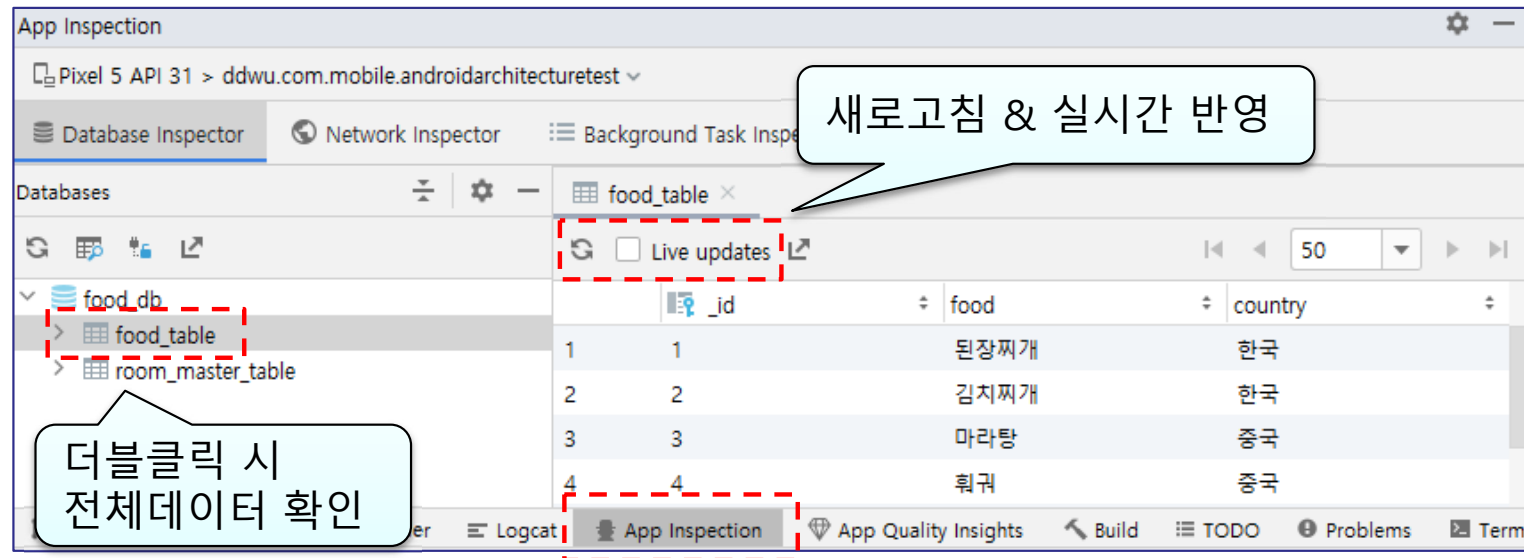
```
val foodRepo = (application as FoodApplication).foodRepo
val foodFlow : Flow<List<Food>> = foodRepo.allFoods
```





# 참고 사항

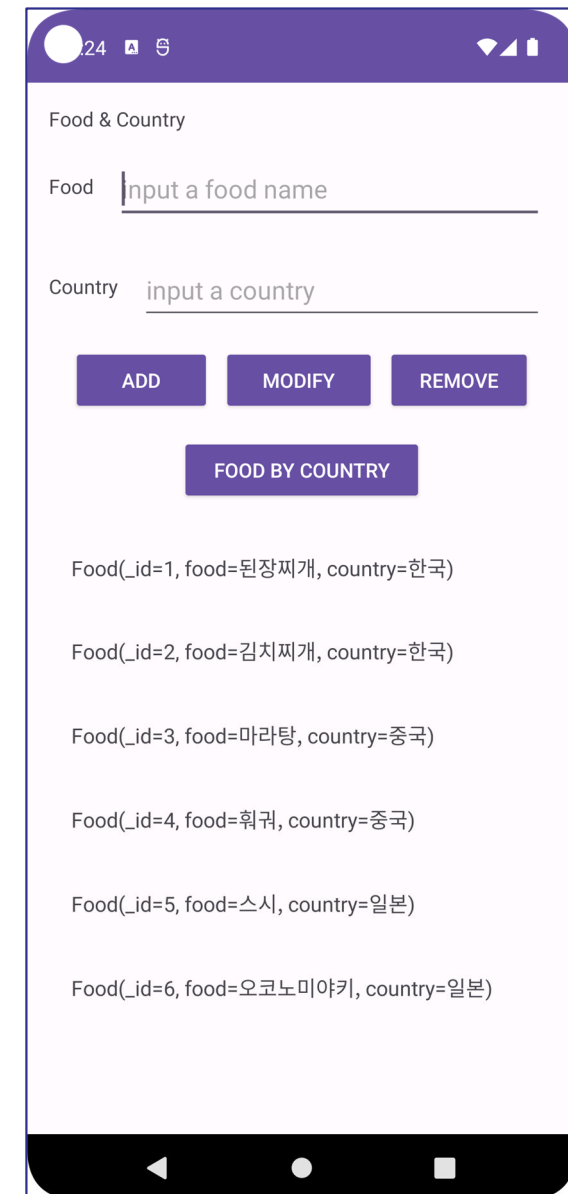
## Database Inspector



# 실습 1

다음 조건에 해당하는 앱을 구현하시오.

- ◆ RoomExam01 사용
- ◆ Modify(Update) 구현
  - Food를 기준으로 Country 변경
- ◆ Remove(Delete) 구현
  - Food를 기준으로 삭제
- ◆ Remove 구현 후 RecyclerView 항목 롱클릭 시 삭제로 변경



# 참고

---

## 앱 아키텍처 가이드

- ◆ <https://developer.android.com/topic/architecture?hl=ko>

## 데이터 레이어 (Repository)

- ◆ <https://developer.android.com/topic/architecture/data-layer?hl=ko>

## ViewModel 개요

- ◆ <https://developer.android.com/topic/libraries/architecture/viewmodel?hl=ko>

## 뷰를 사용한 Android Room – Kotlin

- ◆ <https://developer.android.com/codelabs/android-room-with-a-view-kotlin?hl=ko#0>

